CS781 Capstone Project: In-House Legal Intelligence System

Kalyan Babu Kakarapathi
Roll No. 261110402
khlvn26@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

I investigated a legal-language-model pipeline spanning seven tasks, from domain-specific pretraining to structured extraction, document classification, summarization and retrieval. A decoder-only transformer was pretrained on a controlled mixture of official legal corpora and extended to named-entity recognition, rhetorical-role classification, judgment prediction, statute identification, abstractive summarization and prior-case retrieval. The downstream systems examine complementary forms of adaptation, including supervised fine-tuning, low-rank updates, knowledge distillation and sparse–dense retrieval. Evaluation is conducted with task-specific metrics and public development data. The experiments illustrate the importance of preserving document structure, matching prediction formats to evaluation criteria and calibrating decisions for imbalanced labels. They also identify a case in which a trained neural retrieval component did not improve upon a lexical baseline. Codabench results are distinguished from local development-set results because they measure different evaluation splits.

1 Competition Result

Codabench username: k_261110402 **Roll number:** 261110402

Table 1: Codabench performance by phase on 9 November 2026. Scores, ranks and submission identifiers are shown for the highest-scoring completed submissions.

Phase	Task	Primary metric	Score	Submission ID	Rank	Finished
1	Legal language modelling	BERTScore F1	**0.8090**	963992	**132**	6
2	Legal NER	Strict Macro-F1	**0.1915**	974914	**128**	8
3	Rhetorical-role prediction	Macro-F1	**0.3312**	993925	**129**	5
4	Court judgment prediction	Macro-F1	**0.5746**	1008152	**125**	5
5	Legal statute identification	Macro-F1	**0.2044**	1013449	**131**	4
6	Abstractive summarization	BERTScore F1	**0.8025**	1023369	**125**	6
7	Prior-case retrieval	Micro-F1@7	**0.2806**	1037975	**107**	4

Completed scored submissions: 38 across seven phases. **Results dated:** 9 November 2026.
Code repository: K-Kalyan-B/CS781-Capstone-Project.

The table includes completed submissions only. Phase 6 had three submissions tied at 0.8025, and all four completed Phase-7 submissions scored 0.2806. Subsequent tables compare the individual model variants on their respective local development evaluations; these are separate from the official leaderboard measurements.

2 Introduction and Problem Description

Legal judgments require more than general-purpose text classification. A judgment may combine factual background, procedural history, submissions, statutory provisions, citations and judicial reasoning within a single lengthy document. These elements are related, but they correspond to different prediction units: character spans, sentences, whole cases, generated text and ranked documents. This project studies whether representations acquired through legal-language pretraining can support these heterogeneous tasks under a common data and compute budget.

The work follows the seven-task sequence established in the CS781 capstone [1]. Phase 1 generates a 128-word continuation from a legal-text prompt. Phase 2 extracts exact character offsets and labels for twelve entity types. Phase 3 assigns rhetorical roles to sentences from Corporate Law (CL) and Income Tax (IT) judgments. Phases 4 and 5 predict, respectively, a binary case outcome and a set of applicable statute identifiers. Phase 6 generates an abstractive case summary, whereas Phase 7 ranks seven relevant prior cases for a query document. The official evaluation criteria range from strict span Macro-F1 to BERTScore F1 and Micro-F1@7; consequently, improvements within one phase cannot be interpreted using the numerical scale of another.

The investigation is framed by three questions. First, how can a compact language model be trained from the permitted legal sources without third-party pretrained weights? Second, which task representations and adaptation strategies are practical when legal documents exceed the available context window? Third, what can development metrics establish about structured prediction, generation and retrieval, and where are additional controls needed? The foundation-model size is constrained to 50–300 million parameters; external legal corpora and commercial model APIs are excluded, and the implementations target Kaggle/Colab-class GPU environments. The study concerns independently evaluated task models, not a deployed legal-advice service. Its broader motivation is consistent with the Indian legal NLP benchmark literature [2].

3 Data Analysis and Preprocessing

3.1 Official corpus and task datasets

All experiments use the authorized CS781 capstone dataset [3]. The language-modelling corpus draws on Supreme Court judgments, High Court judgments, Acts and Tribunal documents. Table 2 shows the token allocation encoded in the Phase-1 notebook. After construction of fixed-length sequences, the saved execution records 149,999,616 training tokens and 3,999,744 validation tokens, rather than simply assuming that the nominal target was reached.

Table 2: Phase-1 legal-corpus mixture and token budget, as recorded in the training notebook.

Source	Mixture weight	Training tokens (M)	Validation tokens (M)
Supreme Court	40%	60.0	1.6
High Courts	25%	37.5	1.0
Acts	20%	30.0	0.8
Tribunals	15%	22.5	0.6
Total	100%	150.0	4.0

The supervised phases differ materially in supervision density and label distribution. Phase 2 is based on annotated entity spans in the supplied folds; the retained fold-1 and fold-2 processing records 4,120 and 4,273 annotated spans, respectively. On fold 2, only 2,953 of 4,273 gold spans

(69.1%) start and end at exact BPE token boundaries; this is an annotation–tokenization mismatch, not a measured model error rate. Phase 3 contains 16,845 training and 2,142 development sentences. Its thirteen rhetorical-role classes are unevenly represented: *RatioOfTheDecision* and *Fact* have 4,301 and 4,098 training sentences, while *PrecedentOverruled* has three. This motivates macro-averaged metrics and class-aware training rather than relying on accuracy alone (Figure 1).

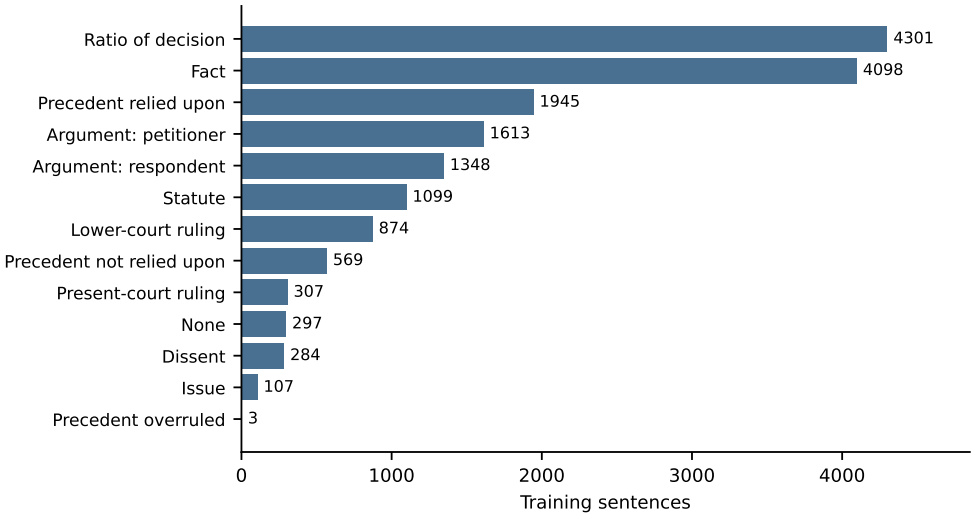


Figure 1: Rhetorical-role frequency in the recorded Phase-3 training data, pooling the CL and IT subsets.

For Phase 4, the retained training audit reports 31,652 labelled cases (18,539 in class 0 and 13,113 in class 1), after removing 653 redundant or contradictory exact-text copies. The 994-case development set is evenly split between classes. In Phase 5, the recorded splits contain 42,750 training, 10,181 development and 13,019 test cases, covering 100 statute IDs. Of the training cases, 33,604 (78.6%) have multiple labels, making independent class decisions necessary. The Phase-6 notebook divides 7,030 document–summary pairs into 6,327 training and 703 validation cases. Phase 7 provides 118 development queries with 973 reference links in total (8.25 links per query on average). These figures describe the supplied notebooks' particular splits and are not claims about unreleased test annotations.

3.2 Text representation and preprocessing

The Phase-1 tokenizer uses a 32,000-entry byte-level BPE vocabulary. Byte-level segmentation permits uncommon names, abbreviations, statutory citations and punctuation to be represented without requiring a manually curated legal lexicon. For exact named-entity evaluation, the original document text and character coordinates are retained throughout preprocessing. Phase 2 generates proposals in overlapping 768-token windows with a stride of 576 tokens, and decodes predicted spans back into original character offsets; this avoids treating token boundaries as necessarily identical to annotated entity boundaries.

The remaining tasks operate at increasingly coarse granularities. Phase 3 preserves the official sentence order and augments sentence classification with neighboring context. Phase 4 forms case representations from up to three 256-token passages; Phase 5 uses up to three 192-token chunks for multi-label prediction. Phase 6 casts a document and its summary as a conditional generation pair and excludes source tokens from the supervised target loss. Phase 7 prepares lexical features from candidate texts and neural features from two 160-token chunks per document. These explicit

truncation and aggregation policies are essential methodological details because the judgments can be substantially longer than a single input window.

4 Approach and Model Evolution

4.1 Phase-1 foundation model

The main Phase-1 backbone is a 15-block grouped-query decoder trained by causal language modelling. The present source defines 15 blocks, hidden dimension 512, eight attention query heads, four key/value heads, a 2,048-token context and tied input/output token embeddings. Grouped-query attention, rotary positions, RMS normalization and SiLU-based feed-forward layers are used within a pre-normalized decoder. The design follows the broad transformer framework [4], but the network is initialized and trained on the official legal corpus rather than loaded from an external language-model checkpoint. For tokens $x_1, \ldots, x_T$, the training objective is

$$\mathcal{L}_{\mathrm{LM}}(\theta) = -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}). \tag{1}$$

The grouped-query decoder has **59,665,152 trainable parameters (59.67M)** across 15 transformer layers at width 512, meeting the required 50–300M parameter range. A run with this architecture recorded validation cross-entropy of 5.1199. Training uses 2,048-token blocks, gradient accumulation and causal language-model supervision. At inference, the notebook samples autoregressively with temperature 0.2 and nucleus probability 0.85, caps generation at 384 tokens, and targets 128 output words.

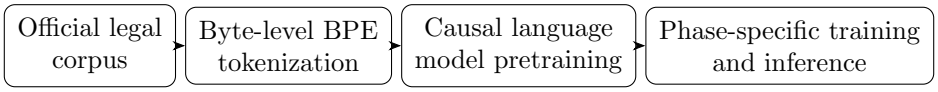


Figure 2: Training and adaptation workflow. Each downstream experiment defines its own task model; Model-4-based phases use a separate, compatible pretrained backbone.

4.2 Transfer to downstream tasks

The phase-specific systems are descendants of legal-language pretraining rather than components of one jointly trained multitask model. A compatible Phase-1 tokenizer and checkpoint are loaded before supervised adaptation. The recorded final configurations use the Model-2 backbone in Phases 2, 4, 6 and 7, and a separately trained Model-4 backbone in Phases 3 and 5. This distinction matters: matching tokenizers and checkpoint architecture are prerequisites for valid weight transfer. Adaptation strategies include full-backbone optimization, parameter-efficient low-rank updates and distillation from a task-trained teacher [5, 6].

4.3 Structured recognition and classification (Phases 2–5)

In Phase 2, a character-aware span classifier predicts entity start/end positions and one of the twelve official entity labels. Candidate generation, span scoring and document-level overlap resolution are treated as separate stages. The system is evaluated against exact triples $(\text{start}, \text{end}, \text{label})$, making alignment preservation a functional requirement rather than a preprocessing convenience.

Phase 3 uses a fine-tuned Model-4 sentence classifier with left-context features and class-weighted supervision. The sentence logits are subsequently combined with a transition model estimated from the labelled training sequences. The transition strength is selected using development labels; this

is a sequence-decoding operation rather than additional language-model pretraining. CL and IT records are trained jointly, while domain-specific scores are examined separately.

Phase 4 predicts a binary case outcome from representations of several passages, aggregated into a document-level classifier. Phase 5 instead assigns a vector of statute probabilities to each judgment. Its student network is trained using both reference labels and a larger teacher model. Because a case may reference multiple statutes, the prediction head uses independent multi-label decisions, with threshold calibration conducted on development data. These phases exemplify the difference between document-level single-label and multi-label supervision.

4.4 Generation and retrieval (Phases 6–7)

Phase 6 adapts a Model-2 causal decoder with LoRA for document-conditioned summary generation. Only the summary tokens contribute to the supervised generation loss. The notebook records both overlap-based evaluation (ROUGE) and a separate embedding-similarity evaluation (BERTScore), with the latter computed on a restricted validation sample [7, 8]. Generation remains abstractive; no external summarization service is used.

For Phase 7, lexical retrieval first orders candidate judgments using word- and character-level TF–IDF. A partially fine-tuned Model-2 encoder computes semantic scores for a limited candidate shortlist. Their fusion is formulated as

$$s_\lambda(q,d) = (1-\lambda)s_{\text{lex}}(q,d) + \lambda s_{\text{neural}}(q,d), \quad 0 \le \lambda \le 1, \tag{2}$$

where λ is chosen only on labelled development queries. The final prediction lists the seven highest-ranked unique candidate IDs for each query. This arrangement permits an explicit comparison between sparse ranking and a neural addition under the same candidate-pool restriction.

5 Experiments and Implementation Details

5.1 Training configuration and reproducibility

The experiments were executed as Python/PyTorch notebooks in GPU-enabled environments. The included files retain training logs, evaluation cells and submission-export procedures. The recorded Model-2 pretraining used 2,048-token sequences and one completed epoch; supervised runs employed gradient accumulation and task-specific early stopping or checkpoint restoration. Table 3 reports the main configuration values read from the corresponding notebooks. It is not a proposed hyperparameter search space.

Table 3: Training configurations recorded in the phase notebooks; official Codabench results are reported separately.

	P1	P2	P3	P4	P5	P6	P7
Backbone	M2	M2	M4	M2	M4	M2	M2
Adaptation	Pretrain	Full	Full	Full	Distil	LoRA	Partial
Batch size	2	1	2	2	4	1	2
Grad. accum.	16	8	8	4	2	4	8
Max. epochs	1	12	6	4	3	3	3
Input budget	2048	768	256	3×256	3×192	640	2×160

Downstream models require architecture-compatible foundation weights and tokenizers. No checkpoints are bundled, and the documented notebooks alone are not a complete reproduction of trained predictions. Data are obtained from the authorized course source, and environment-specific

working paths must be adjusted. An executed run of the 59.67M grouped-query decoder records one epoch with training and validation cross-entropy of 4.6083 and 5.1199, respectively. A wider experiment produced a separate log, which was kept out of the compact-model results.

Phase-specific controls. The Phase-2 span model uses 768-token overlapping windows (576-token stride), batch size 1 with eight accumulation steps and up to 12 epochs. Its recorded backbone and task-head learning rates are 3×10^{-6} and 3×10^{-5}, respectively. Phase 3 uses a maximum 256-token input and batch size 2 with eight accumulation steps; dropout is 0.15, and backbone/head learning rates are configured separately. Phase 4 selects a development threshold of 0.398406 at its best epoch (epoch 2) rather than assuming a fixed 0.5 threshold. The Phase-5 distillation notebook records temperature $T = 2$ and a final decision rule combining a global threshold of 0.35 with 91 per-label thresholds. Phase 6 uses LoRA rank 8 and scaling $\alpha = 16$; only 430,080 of 60,095,232 model parameters are trainable (about 0.716%). In Phase 7, 11,674,368 parameters are trainable; the lexical shortlist contains 60 candidates before selecting the final seven, with a recorded CUDA peak allocation of 519,344,128 bytes. These settings describe the local training runs; leaderboard outcomes are summarized separately in Table 1.

5.2 Local evaluation

Phase 2 reserves the second annotated fold for strict-span evaluation. Phases 3–5 and 7 use their provided development labels for checkpoint or threshold selection. Phase 6 uses a document-ID-level validation split; its saved BERTScore run covers 30 generated summaries rather than the complete validation subset. Metrics are computed at the task's official prediction unit: entities, sentences, cases, text continuations or ranked candidates. Official Codabench metrics are transcribed from completed submissions, while the notebook diagnostics below use the stated local splits.

5.3 Ablations and controlled comparisons

Within-phase comparisons are restricted to settings with identifiable evaluation protocols. The Phase-3 notebook reports both raw classifier predictions and sequence-decoded outputs on the same development set. Phase 7 logs the lexical retrieval baseline and the interpolated system, with λ chosen on development data. Other adaptation notebooks provide useful comparative observations, but differences in training configuration and preprocessing mean they should not be represented as controlled, single-factor ablations.

6 Results

The competition scores in Table 1 come directly from the seven supplied Codabench screenshots. Table 4 instead records the local metrics saved inside the seven retained notebooks. Their evaluation sets and, potentially, checkpoint versions differ; the numbers should not be subtracted to infer a development-to-test generalization gap.

Table 4: Local notebook evaluation, reported separately from Codabench. The seven retained notebooks are provided in the code archive.

Phase	Retained method	Metric	Local value	Scope
1	Grouped-query causal decoder	Validation CE	5.1199	Architecture-matched run
2	Character-span classifier	Strict Macro-F1	0.2339	Fold 2
3	Full FT and transition decoder	Macro-F1	0.3518	CL + IT dev
4	Multi-passage full FT	Macro-F1	0.5924	Development
5	Teacher–student distillation	Macro-F1	0.2176	Development
6	LoRA conditional generation	BERTScore F1	0.8060	30-case subset
7	Partial fine-tuning and retrieval	Micro-F1@7	0.3013	118 queries

Phase-1 local evaluation. A completed run of the 15-block, width-512 grouped-query decoder records **59,665,152 parameters**, training cross-entropy **4.6083**, and validation cross-entropy **5.1199** after one epoch (label smoothing 0.1). These measurements characterize an earlier run of the same architecture, rather than the exact checkpoint later used for submission. The separate generation-based Codabench assessment appears in Table 1. The other new Phase-1 notebooks confirm a 62.01M-parameter encoder–decoder (with resumed training reported complete) and a 53.22M-parameter attention-window/sink design (with a saved checkpoint loaded to generate 1,000 continuations). Their outputs demonstrate execution but do not provide a directly matched validation cross-entropy.

Two notebook-level diagnostics are particularly useful. First, the Phase-3 role classifier obtained Macro-F1 0.2996 before transition decoding and 0.3518 afterward on the same development set; the decoded CL and IT scores were 0.4277 and 0.2057, respectively. Second, the Phase-2 candidate generator covered 2,776 of the 4,273 annotated fold-2 spans (65.0%) before final span classification. Both measurements identify specific components that can be inspected further without conflating backbone size with postprocessing.

6.1 Within-phase comparisons from saved notebooks

The project archive contains at least four distinct approaches for every phase. Tables 5 and 6 place their recorded local results side by side. Because some experiments change multiple components at once—tagging, backbone, preprocessing and training schedule—the comparisons describe complete recipes rather than isolated single-factor effects. In particular, the Phase-3 raw/decoded pair is the same model, while the other rows should be read as implementation comparisons.

Table 5: Within-phase local validation comparisons from notebook outputs; not Codabench scores. Phase 2 uses strict span Macro-F1; Phases 3–5 use task-specific Macro-F1. Bold marks the strongest *listed local* run in each phase.

Phase	Training and prediction strategy	Local F1
2	Character-aware span extraction (grouped-query backbone)	**0.2339**
2	BIOES tagging with low-rank adaptation	0.1616
2	Knowledge-distilled BIOES tagging	0.1159
2	Full fine-tuning with BIO token labels	0.0779
3	Windowed backbone: full fine-tuning + transition decoding	**0.3518**
3	Same classifier, before transition decoding	0.2996
3	Grouped-query backbone: knowledge distillation	0.2874
3	Windowed backbone: LoRMA adaptation	0.2583
3	Grouped-query backbone: balanced full fine-tuning	0.2501
4	Multi-passage, grouped-query full fine-tuning	**0.5924**
4	Windowed backbone: knowledge distillation	0.5866
4	Windowed backbone: LoRMA adaptation	0.5448
4	Grouped-query backbone: LoRA adaptation	0.5376
5	Windowed backbone: knowledge distillation	**0.2176**
5	Grouped-query backbone: full fine-tuning	0.1966
5	Windowed backbone: LoRMA adaptation	0.1530
5	Grouped-query backbone: LoRA adaptation	0.0867

Pretraining, summarization and retrieval variants. Table 6 gives four distinct implementations for each of Phases 1, 6 and 7. In Phase 1, two decoder runs record validation cross-entropy, while the encoder–decoder and attention-window designs contribute architecture sizes and execution evidence. Phase-6 ROUGE-L diagnostics were not all computed on a verified identical subset and should not be ranked as a controlled ablation. In Phase 7, all four recorded systems have the same local Micro-F1@7 because development tuning assigns zero weight to the neural score, so the tie follows from the development-tuned lexical fusion.

Table 6: Further within-phase comparisons. Phase 1 contrasts model size and training evidence, with validation cross-entropy given for the two runs that logged it. Phase 6 reports diagnostic ROUGE-L values from different subsets; Phase 7 uses Micro-F1@7 on 118 development queries.

Phase	Approach	Size / method	Local result
1	Baseline decoder (15 blocks, width 512)	59.65M	Val CE 6.0278
1	Grouped-query decoder (15 blocks, width 512)	**59.67M**	Val CE 5.1199
1	Encoder–decoder transformer	62.01M	Training completed
1	Attention-window and sink decoder	53.22M	1,000 outputs
6	Grouped-query backbone: LoRA adaptation	r=8	ROUGE-L 0.1248
6	Pfeiffer adapter	Adapter	ROUGE-L 0.1136
6	Knowledge distillation	Student	ROUGE-L 0.1128
6	LoRMA variant	Low rank	ROUGE-L 0.1691
7	Grouped-query backbone: full fine-tuning	Full FT	F1@7 0.3013
7	Grouped-query backbone: partial fine-tuning	Partial	F1@7 0.3013
7	Windowed backbone: LoRMA adaptation	LoRMA	F1@7 0.3013
7	Windowed backbone: distillation	KD	F1@7 0.3013

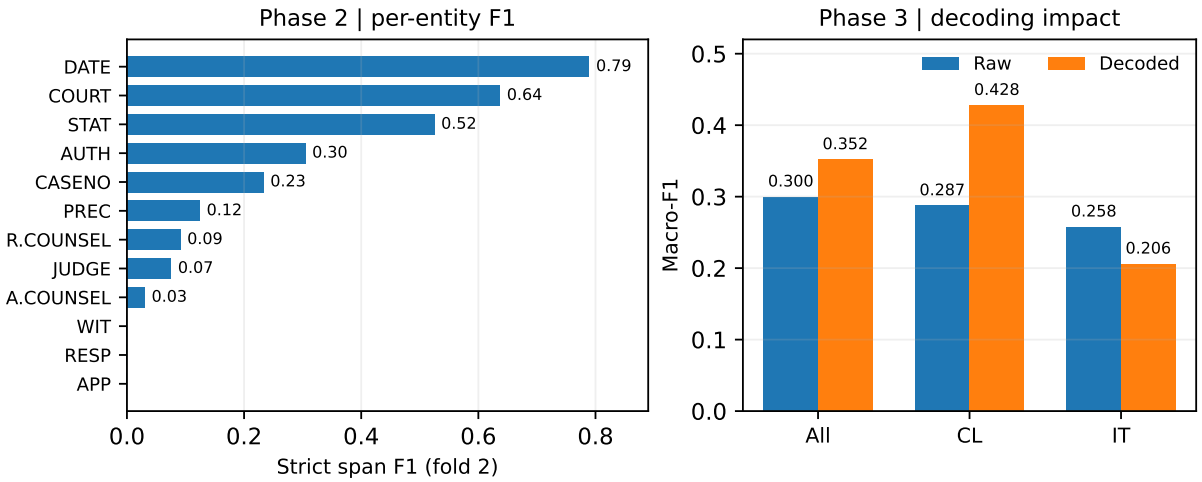


Figure 3: Saved development diagnostics. (Left) Phase-2 strict-span F1 by entity class at epoch 12 (fold 2). (Right) Phase-3 raw and transition-decoded Macro-F1; the decoder helps on the combined set and CL but not IT.

Observed classification errors. On the balanced 994-case Phase-4 development set, Model 2 predicted 332 true negatives and 259 true positives, alongside 165 false positives and 238 false negatives (threshold 0.398406). The Phase-2 fold-2 exact-span counts were TP=1,585, FP=1,692 and FN=2,688. These are evaluation counts, not reconstructed case examples.

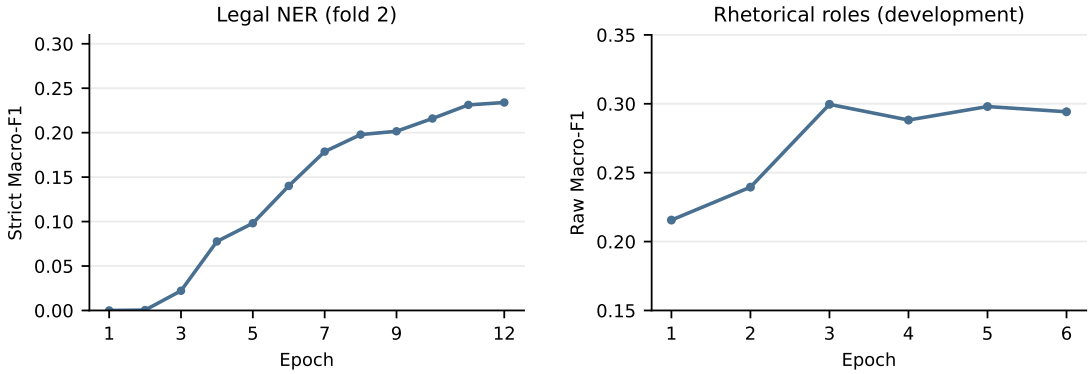


Figure 4: Recorded local training trajectories. Different panels use different task metrics; they do not represent Codabench submission histories.

Phase 6 BERTScore F1 0.8060 was evaluated on 30 generated summaries, so it remains a local diagnostic. In Phase 7, the top-60 lexical shortlist has oracle F1@7 0.5181, an upper bound under that candidate restriction, not an achieved ranking score. The Phase-5 student records Micro-F1 0.2900 versus Macro-F1 0.2176, reflecting the distinction between micro- and macro-averaged multi-label evaluation.

7 Error Analysis and Discussion

7.1 Span boundaries and label balance

Strict-span recognition is sensitive to character offsets, which is why Phase 2 keeps the original text and aligns predictions back to document positions. Its local candidate-coverage diagnostic is useful because missing a gold candidate at proposal time cannot be repaired by the subsequent type classifier. In rhetorical-role prediction, *RatioOfTheDecision* and *Fact* are frequent while *PrecedentOverruled* is almost absent from training. This imbalance makes Macro-F1 and domain-wise inspection more informative than overall sentence accuracy. For statute identification, independent sigmoid outputs fit the multi-label target, but predicting rare labels remains sensitive to the tuned thresholds.

7.2 Long-document modelling and retrieval

Judgments exceed the available context lengths, so the document-level models aggregate representations of multiple passages rather than relying on a single opening window. Such selection is a practical compromise, not proof that the chosen passages always contain the decisive facts. The retrieval notebook provides a particularly clear control: after tuning the lexical–neural interpolation on development queries, the optimized setting retained the lexical ranking. A controlled dense-reranking evaluation under a fixed shortlist would be needed to test whether the neural representation helps on other queries.

7.3 Limitations and next steps

The most consequential limitations are incomplete coverage of long judgments, highly uneven supervision for rare categories, and restricted validation samples for generation. Automatic text-overlap and embedding-similarity metrics cannot by themselves establish whether a summary preserves the holding, statutory references and important qualifications. Useful follow-up checks would be a targeted audit of NER boundary errors, per-role and per-statute confusion tables, matched passage-budget comparisons for document classification, and human review of a small stratified set of generated summaries. These tests address identifiable weaknesses in the models rather than attempting to infer legal reliability from one aggregate score.

8 Conclusion

This project covers seven legal-NLP tasks with a foundation model trained on the permitted corpus and task-specific prediction heads. Its strongest experimental lessons come from the way the output is represented: exact character alignment for entity extraction, sequence-aware decisions for rhetorical roles, passage aggregation for long documents and calibrated thresholds for multi-label statutes. The development experiments also show that adding a neural retrieval score did not improve the local ranking. Beyond the official competition results, further work would benefit most from controlled comparisons of these components and a factuality check of generated legal summaries.

LLM Usage Declaration

Q1. Which Large Language Model(s), if any, were used?
OpenAI ChatGPT.

Q2. Where was an LLM used?

Part	Used?	Purpose and scope
Understanding concepts / papers	Yes	Explained components and methods discussed in research papers.
Ideation / brainstorming at a high level	Yes	Compared possible building blocks; I made the final design decisions.
Debugging small code snippets	Yes	Helped resolve Python, PyTorch and notebook runtime errors.
Plotting / visualization code	Yes	Generated plots and visualizations from my experimental results.
Grammar / writing style of the report	Yes	Polished grammar and presentation using my rough outline.
Report structure / section wording	Yes	Converted my outline and explanations into clearer report prose.
Designing the model architecture	Yes	Explained published components such as SiLU, SwiGLU, sandwich FFNs and attention variants; the architecture design was mine.
Hyperparameter search / tuning	Yes	Suggested configurations and explained trade-offs; I chose and adjusted the hyperparameters.
Analysis of results / conclusions	Yes	Assisted with displaying and visualizing my measurements; I analyzed and interpreted the outcomes.
Writing the core training / inference code	Yes	Generated implementation code based on the architecture, training strategy and experimental choices I specified.
Other (specify)	Yes	Helped turn my rough report outline into polished academic writing.

Q3. Were sources other than lecture slides and an LLM consulted?
I used the official CS781 capstone instructions and datasets, along with the cited research papers on transformers, model adaptation, distillation and evaluation metrics.

Q4. What was the author's contribution?
I designed the foundation-model architectures and decided which attention and feed-forward components to compare. I determined the parameter budgets, data mixture and training settings, drawing on research papers and AI-assisted explanations of relevant components. I planned and carried out the pretraining and downstream experiments using the official datasets. I chose the task formulations, model adaptations, training objectives and evaluation procedures for all seven phases. I directed the implementation using AI-generated Python code based on my design specifications and made the modelling decisions myself. I performed the data checks, compared model variants and interpreted the resulting metrics. For named-entity recognition, I compared token-tagging and character-span approaches, finding that the character-span system handled the strict-offset evaluation more effectively in the recorded experiments. For rhetorical-role prediction, I examined sentence-level predictions and transition-aware decoding across the two legal domains. I developed passage-based classification for judgment prediction and a multi-label strategy for statute identification. For summarization, I evaluated the trained generator using ROUGE and BERTScore. For prior-case retrieval, I tested lexical and neural ranking; neural-score fusion did not improve the development result, so I retained the lexical ranking. I prepared the report outline and substantive experimental interpretation, while using ChatGPT to improve readability and visualize my results.

References

[1] CS781 Course Staff, "CS781 Capstone Project Competition Document," 2026. Course-provided competition instructions.

[2] A. Joshi, S. Paul, A. Sharma, P. Goyal, S. Ghosh, and A. Modi, "IL-TUR: Benchmark for Indian legal text understanding and reasoning," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 11460–11499, Association for Computational Linguistics, 2024.

[3] Exploration Lab, "CS-781 Capstone: Official Phase Data." https://huggingface.co/datasets/Exploration-Lab/CS-781-Capstone, 2026. Dataset used under course restrictions.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[6] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[7] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, pp. 74–81, 2004.

[8] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," *International Conference on Learning Representations*, 2020.